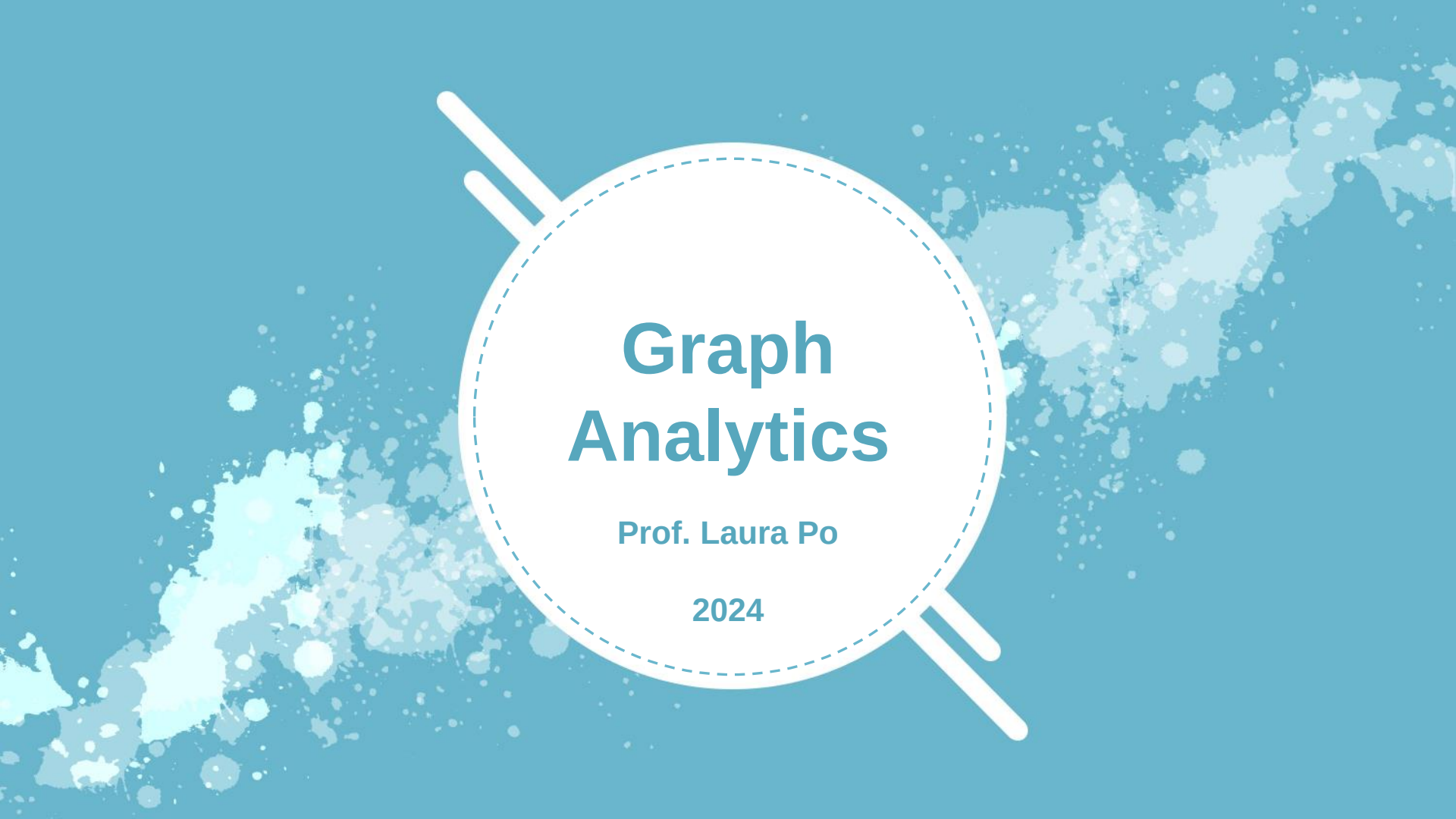




Graph Analytics

Prof. Laura Po

2024

OBJECTIVES

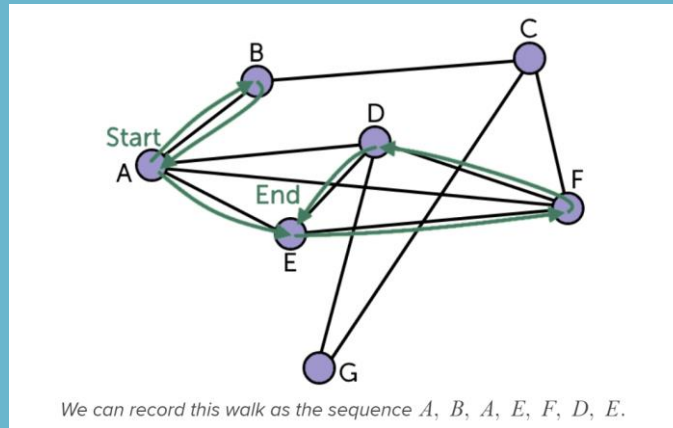
- Understand how to traverse the graph, and find «the best» path (faster / shortest/ cheaper...) among two nodes
- Understand how to calculate statistics on a graphs
- Understand the structure of a graphs

The background is a solid teal color. It features several white, irregular splatters or paint-like marks scattered across the surface. There are also several thin, light blue diagonal lines that appear to be part of a larger graphic design, some in the top left and bottom right corners.

**Walk, Trail,
Circuit, Path,
Cycle**

Walk

- A **walk** is the most general definition of how we can move around a graph.
- If we start at a vertex and then move along an edge to a new vertex, we are starting a walk. If we have a directed graph, we move in the direction of the arrows.
- In a simple graph, we can represent a walk by a sequence of vertices.

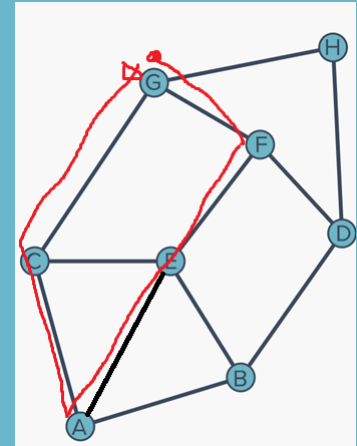
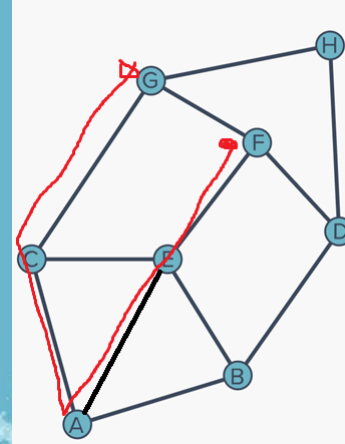
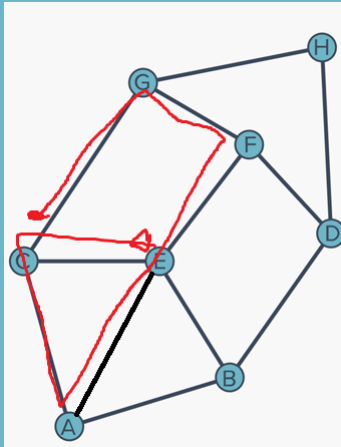
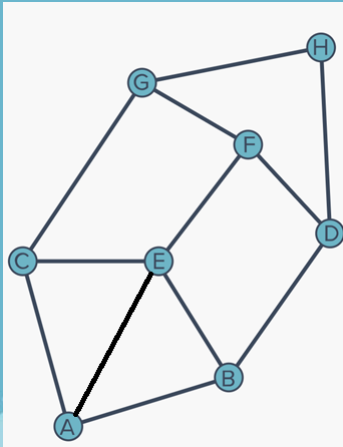


- If a walk starts and ends at the same vertex then it is **closed**, and if it starts and ends at different vertices then it is **open**.

Trail – Path - Cycle

If a walk does not use the same edge more than once, then it is a **trail** - it is an open trail if it's open and a closed trail (also called a **circuit**) if it is closed.

If a walk does not use the same vertex more than once (other than at the start and end) we call it a **path**. When a path begins and ends at the same vertex it is a closed path, which is usually called a **cycle**.



Trails and circuits

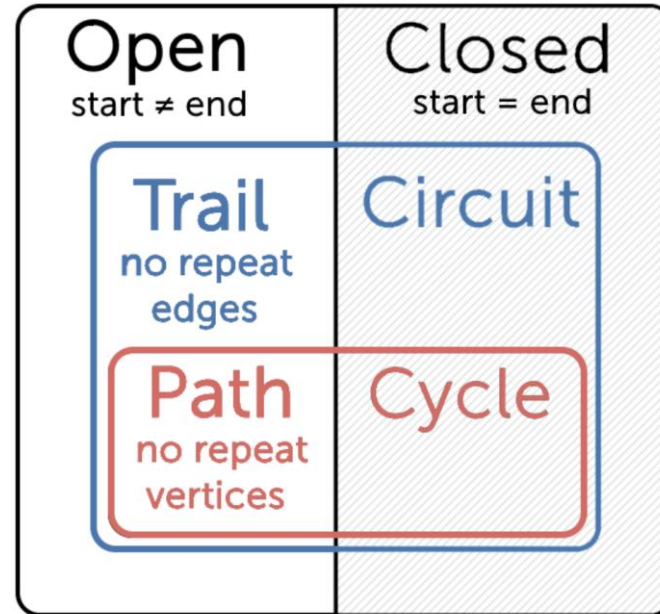
- A trail is a walk where no edge is repeated.
- An open trail starts and ends on different vertices.
- A closed trail (circuit) starts and ends on the same vertex.
- All trails are walks and all circuits are closed walks.

Paths and cycles

- A path is a walk where no vertex is repeated (which also means that no edges can be repeated).
- An open path starts and ends on different vertices.
- A closed path (cycle) starts and ends on the same vertex.
- All paths are walks (and trails) and all cycles are closed walks (and closed trails).

Summary

Walks



1.

Every sequence of edges is a **walk**. A walk can be **open** (start/end vertices different) or **closed** (start/end vertices same).

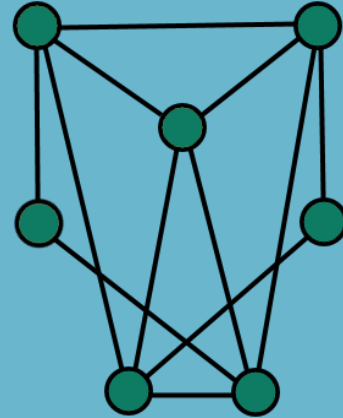
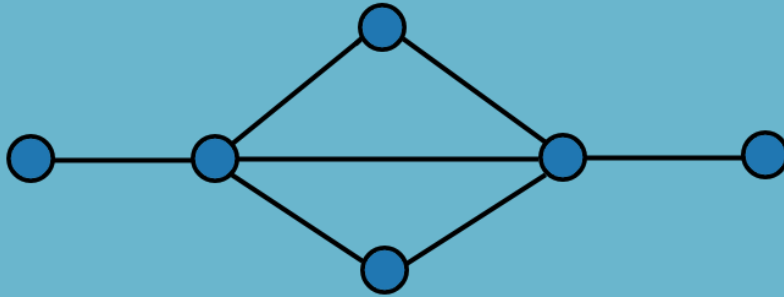
If edges can't repeat, the walk is a **trail**. A closed trail is a **circuit**.

If vertices can't repeat, the walk is a **path**. A closed path is a **cycle**.

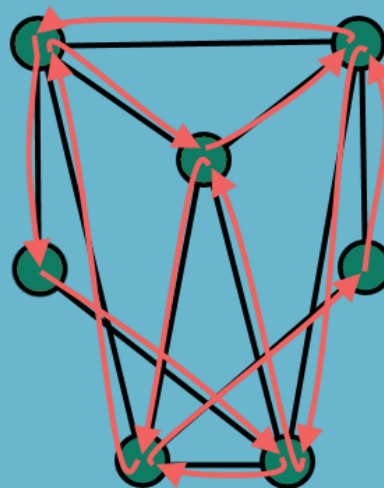
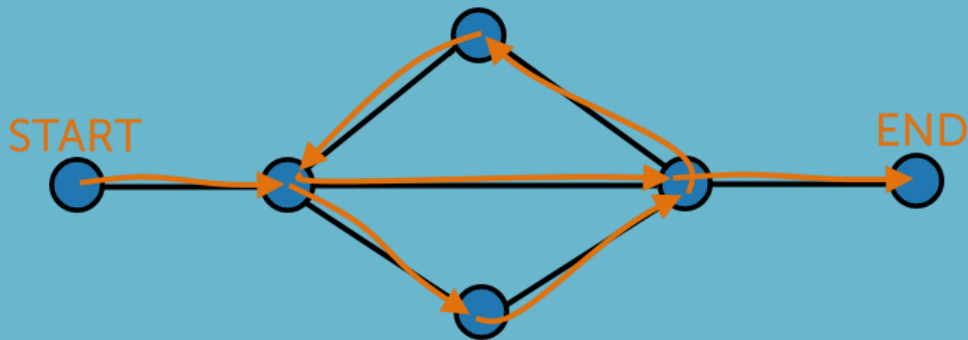
Walks, paths, trails, circuits and cycles **do not** need to use every edge or every vertex in the graph.

Eulerian and semi-Eulerian graphs

- Consider the following graphs. Check that you can start at one vertex and **travel along each edge** exactly once without lifting your pen from the paper.



Eulerian and semi-Eulerian graphs



- Each graph has a trail that uses every edge exactly once. The trail on the left is open and starts and finishes at different vertices. The graph on the right is closed and any vertex can be the start/end.

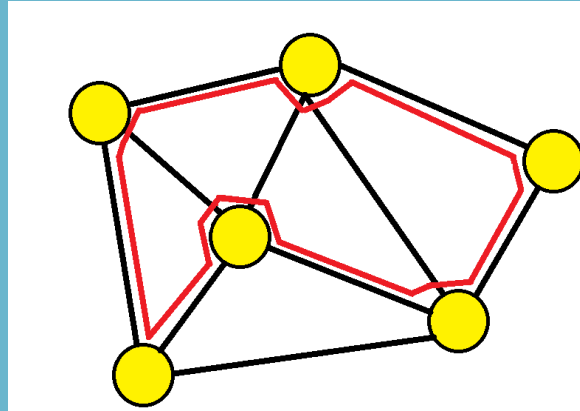
Traversable graphs

Eulerian and semi-eulerian

- **Traversable** is the word to describe graphs where you can start at a vertex and draw in all the edges without taking your pen off the page or retracing any edge.
- The two types of traversable graphs are:
 - If a graph has a closed trail (it starts and finishes at the same vertex) that uses every edge, it is called **Eulerian** (named after the same Euler who gave us the formula $\sum d_i = 2E$). It can also be called an Eulerian trail or an **Eulerian circuit**.
 - If a graph has an open trail (it starts and finishes at different vertices) that uses every edge, it is described as **semi-Eulerian**. It can also be called a semi-Eulerian trail.

Hamiltonian Path

- If there is a path in the connected graph that visits every vertex of the graph exactly once without repeating the edges then such a path is called **Hamiltonian Path**



The background is a solid teal color. It features several white, irregular splatters or paint-like marks scattered across the surface. There are also several thin, light blue diagonal lines running across the background, some parallel and some intersecting.

Graph Navigation

Operations on a graph

What operations we do on a graph?

Given a node obtain:

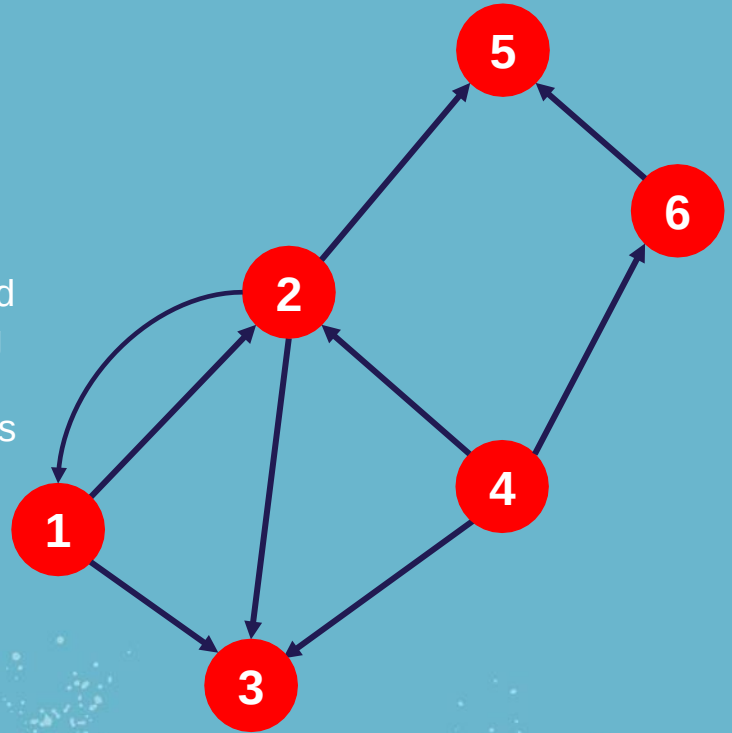
- Neighbors: obtain the list of all nodes connected to it
- Degree: number of nodes connected (when undirected), In-degree /out-degree (when directed)
- Graph Traversal - Start from a node, obtain the list of all reachable nodes

Example

The neighbors of a node in a graph are the nodes that are directly connected to the given node by an edge.

Reachable nodes in a graph are nodes that can be reached from a given starting node by following a path along the edges of the graph. In other words, reachable nodes are those that are connected to the starting node by one or more paths

- $\text{Neighbors}(1) = \{2, 3\}$
- $\text{InDegree}(2) = 2$
- $\text{OutDegree}(2) = 3$
- $\text{Reachable}(4) = \{2, 3, 1, 6, 5\}$
- $\text{Reachable}(1) = \{2, 3, 5\}$

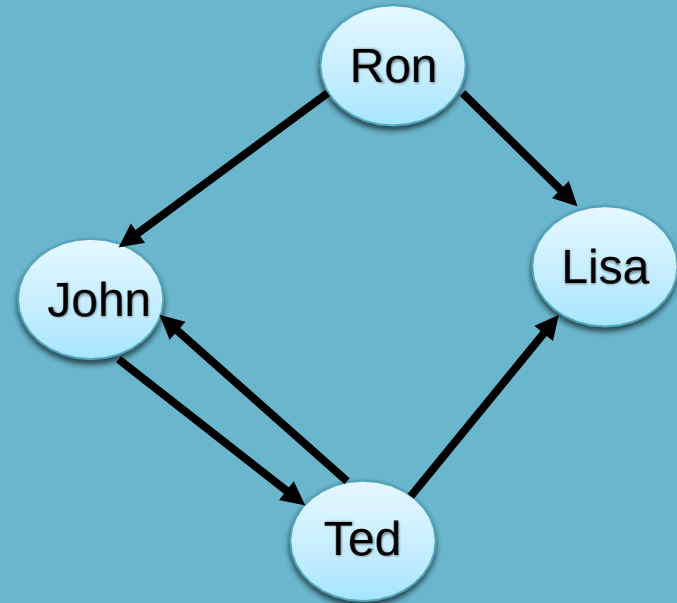


Graph Traversal

Definition

Traverse - The process of moving from vertex to edge or edge to vertex as we navigate through a graph.

“Who are Ted’s friends?” In a relational database, we would use a query to answer this question, but in a graph, we perform a traversal. This process of moving through the graph is known as traversing. The definition of the set of steps and actions we perform to retrieve this data is known as the traversal (traversing a graph), analogous to the act of querying a relational db with an SQL query.

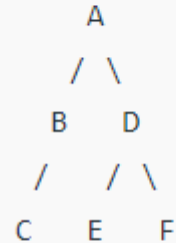


Graph Traversal: DFS

- **Depth First Search:** visit a neighbor of the last visited node
- DFS, Depth First Search, is an edge-based technique. It uses the Stack data structure and performs two stages, first visited vertices are pushed into the stack, and second if there are no vertices then visited vertices are popped.

Example:

Input:



Output:

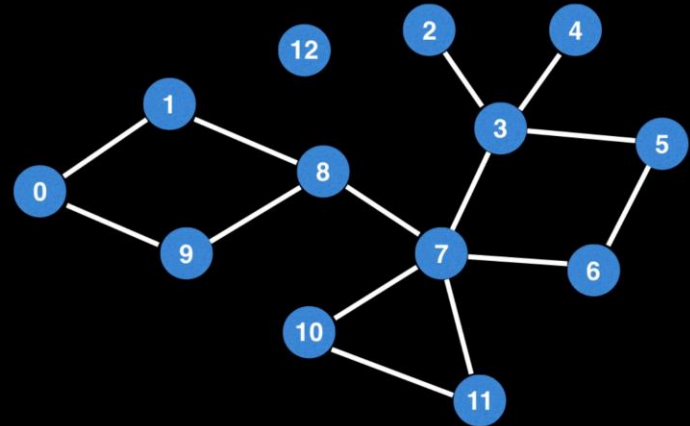
A, B, C, D, E, F

Video

- From 1:04 – till 3:28
- Playback speed 1:75
- <https://www.youtube.com/watch?v=7fujbpJ0LB4&t=68s>

Basic DFS

As the name suggests, a DFS plunges depth first into a graph without regard for which edge it takes next until it cannot go any further at which point it backtracks and continues.

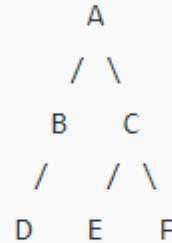


Graph Traversal: BFS

- **Breadth-First Search:** visit first all the neighbors of a node before visiting the other
- BFS, Breadth-First Search, is a vertex-based technique for finding the shortest path in the graph. It uses a Queue data structure that follows first in first out. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue. It is slower than DFS.

Example:

Input:



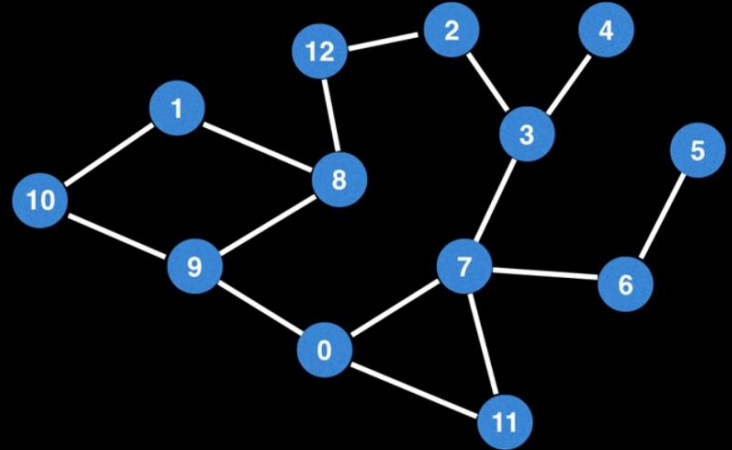
Output:

A, B, C, D, E, F

Video

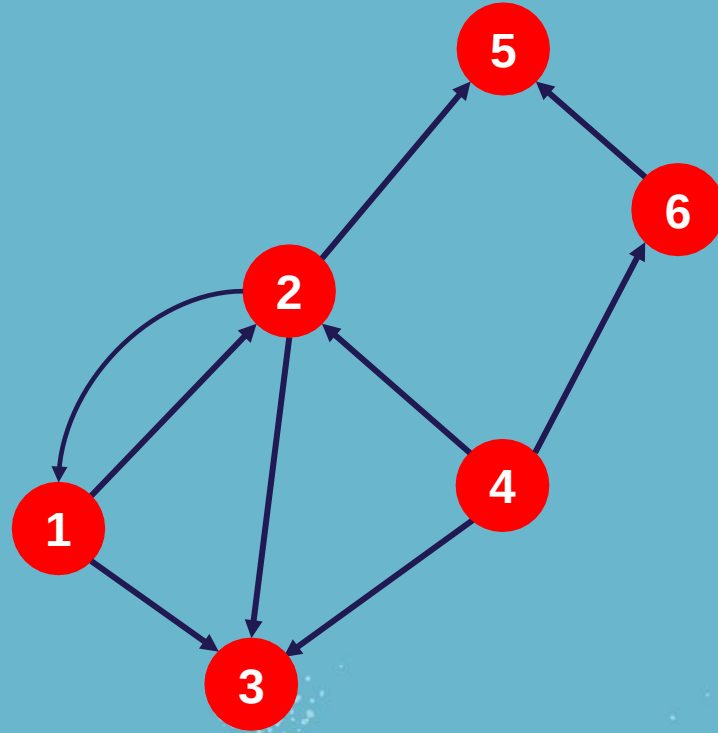
- From 0:56
- Playback speed 1:75
- <https://www.youtube.com/watch?v=oDqjPvD54Ss&t=57s>

A BFS starts at some arbitrary node of a graph and explores the neighbour nodes first, before moving to the next level neighbours.



Example

- Breadth-First Search (BFS): visit first all the neighbors of a node before visiting the other
- $\text{BFS}(4) = [2, 3, 6, 1, 5]$
- Depth-First Search (DFS): visit a neighbor of the last visited node
- $\text{DFS}(4) = [2, 1, 3, 5, 6]$



Graph Traversal: Shortest Path

In graph theory, the shortest path problem is the problem of finding the “quickest” path between two vertices (or nodes) in a graph:

- If all edges cost the same (unweighted graph) → find the smaller number of edges
- If edges have different costs (weighted graph) → find the path such that the sum of the weights of its constituent edges is minimized.

The shortest path problem can be defined for graphs whether undirected, or directed.

Shortest Paths

- **Single source:** Given 1 source node find the “quickest” way to reach all other nodes
- **Single-Pair of nodes shortest path:** given a source and a destination, find the “quickest” way – if exists – between the two
- **All-pairs shortest path:** find the shortest paths between every pair of vertices

<https://www.sixdegreesofwikipedia.com>

BFS & SHORTEST PATH

- If all edges cost equal (unweighted graphs), BFS can compute the shortest path from one node to all other nodes
- BFS → Complexity $O(E + V)$

Algorithms

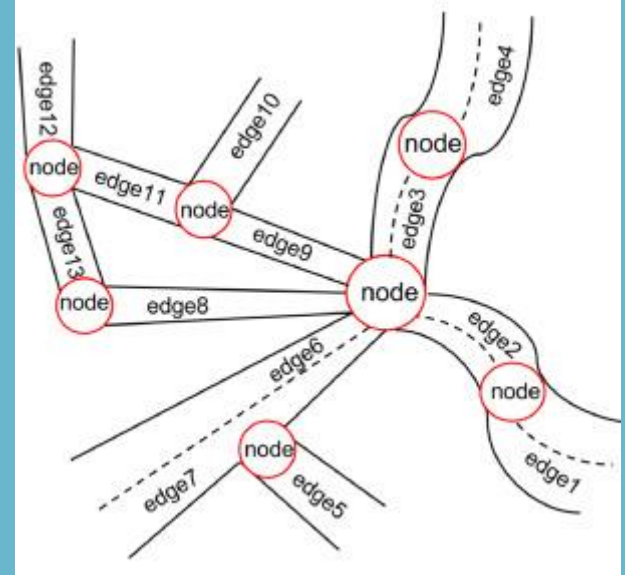
- The most important algorithms for solving this problem are:
- **Dijkstra's algorithm** solves the single-source shortest path problem with non-negative edge weight.
<https://www.youtube.com/watch?v=pSqmAO-m7Lk>
- **Bellman–Ford algorithm** solves the single-source problem if edge weights may be negative.
<https://www.youtube.com/watch?v=lyw4FaxrwHg>
- **A* search algorithm** solves for single-pair shortest path using heuristics to try to speed up the search.
<https://www.youtube.com/watch?v=A60q6dcoCjw>
- **Floyd–Warshall algorithm** solves all pairs shortest paths.
- **Johnson's algorithm** solves all pairs shortest paths, and may be faster than Floyd–Warshall on sparse graphs.
- **Viterbi algorithm** solves the shortest stochastic path problem with an additional probabilistic weight on each node.

Application

- Shortest path algorithms are applied to automatically find directions between physical locations, such as driving directions on web mapping websites like Google Maps. For this application fast specialized algorithms are available.
- If one represents a nondeterministic abstract machine as a graph where vertices describe states and edges describe possible transitions, shortest-path algorithms can be used to find an optimal sequence of choices to reach a certain goal state.

Road network

- A road network can be considered a graph with positive weights.
- The nodes represent road junctions and each edge of the graph is associated with a road segment between two junctions.
- The weight of an edge may correspond to the length of the associated road segment, the time needed to traverse the segment or the cost of traversing the segment.
- Using directed edges it is also possible to model one-way streets.



Road network

- Such graphs are special in the sense that some edges are more **important** than others for long-distance travel (e.g. highways). This property has been formalized using the notion of highway dimension.
- There are a great number of algorithms that exploit this property and are therefore able to compute the shortest path a lot quicker than would be possible on general graphs.
- The algorithm with the fastest known query time is called **hub labeling** and is able to compute shortest path on the road networks of Europe or the US in a fraction of a microsecond

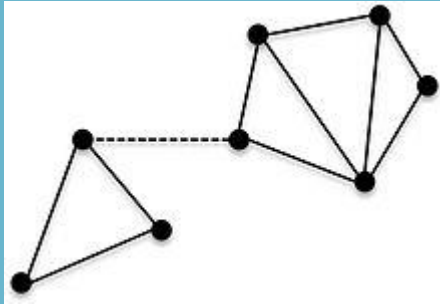
The background is a solid teal color. It features several white, irregular splatters or paint-like marks scattered across the surface. There are also several thin, light blue diagonal lines running across the background, some parallel and some intersecting.

Graph Statistics

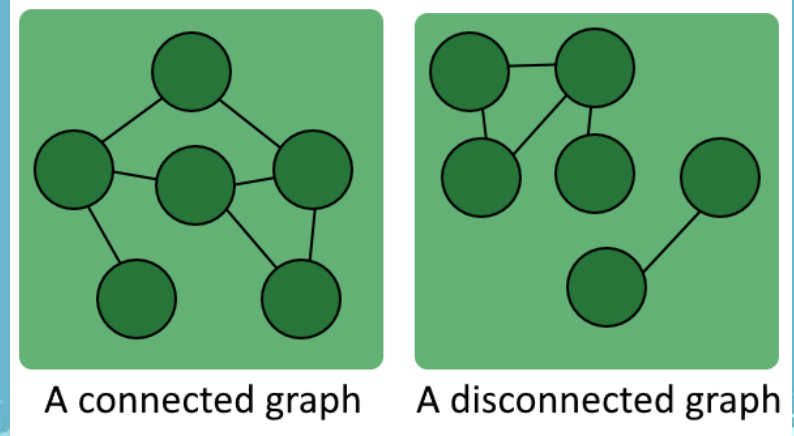
Connectivity

How well the graph is connected

- Connectivity is one of the basic concepts of graph theory: it asks for the minimum number of elements (nodes or edges) that need to be removed to separate the remaining nodes into two or more isolated subgraphs.
- It is closely related to the theory of network flow problems. The connectivity of a graph is an important measure of its resilience as a network.



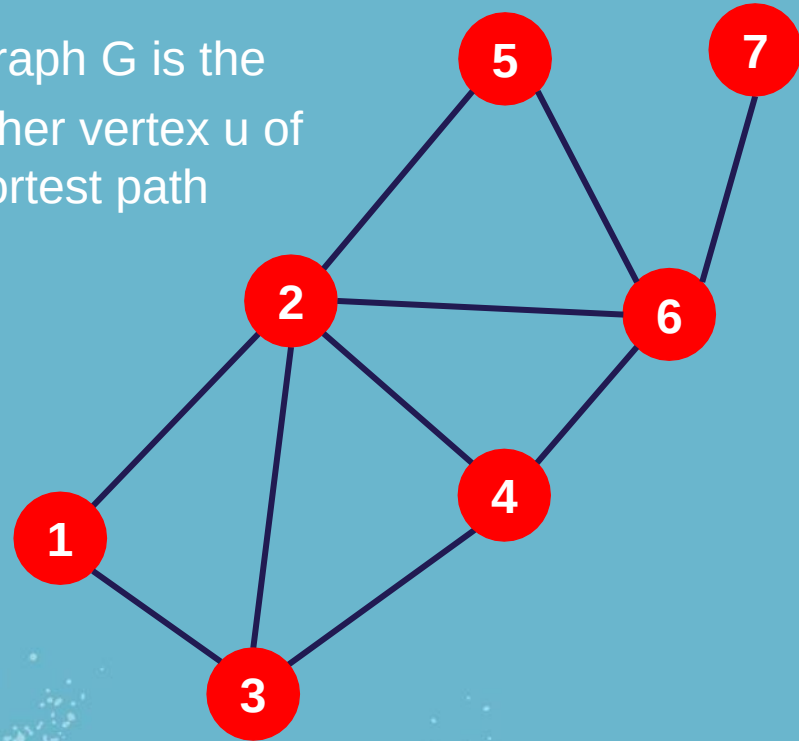
This graph becomes disconnected when the dashed edge is removed.



Statistics

Eccentricity of a node (v) in a connected graph G is the maximum graph distance between v and any other vertex u of G . In another way, the length of the longest shortest path starting at that node.

- Eccentricity(1) = 3
- Eccentricity(2) = 2
- Eccentricity(3) = 3
- Eccentricity(4) = 2
- Eccentricity(5) = 2
- Eccentricity(6) = 2
- Eccentricity(7) = 3



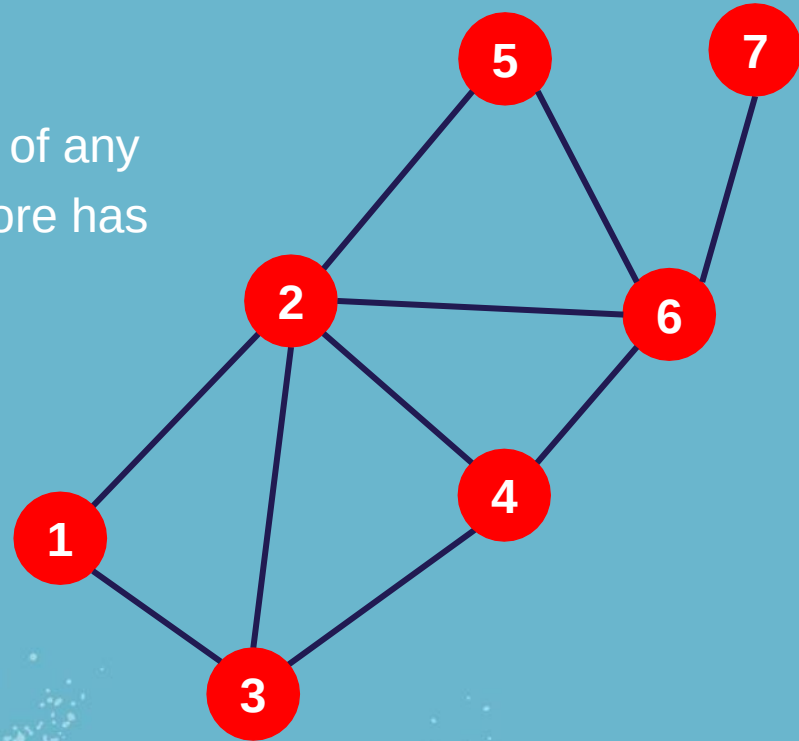
Statistics

Radius of a graph: the minimum eccentricity of any node in the graph [A disconnected graph therefore has infinite radius (West 2000)]

- Radius = 2

Diameter of a graph: the maximum eccentricity of any vertex in the graph, i.e. the maximum distance between any 2 nodes

- Diameter = 3



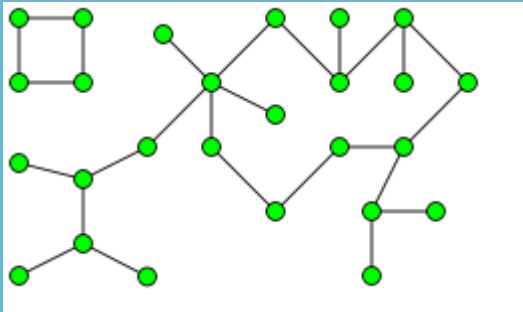
These measures ignore directions of edges

Connected Component

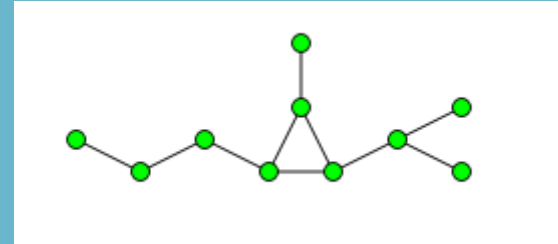
A component, or connected component, of an undirected graph is a connected subgraph that is not part of any larger connected subgraph.

The components of any graph partition its vertices into disjoint sets, and are the induced subgraphs of those sets.

A connected component is a graph partition where each node can reach all other nodes in the same partition.



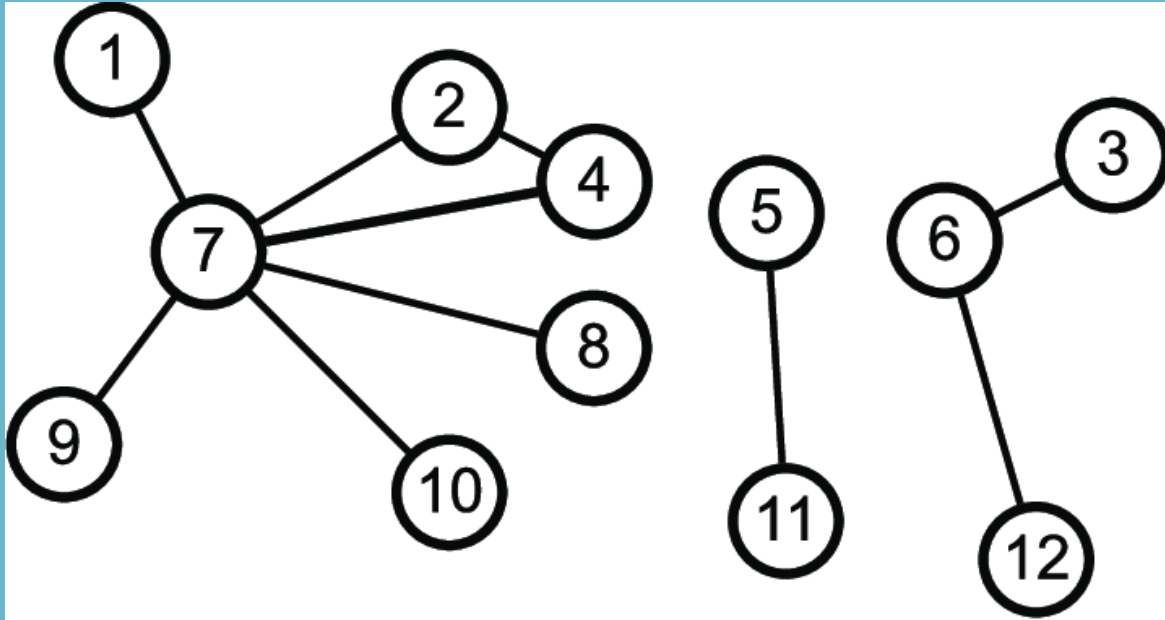
A graph with two component (disconnected graph)



A graph with one component (connected graph)

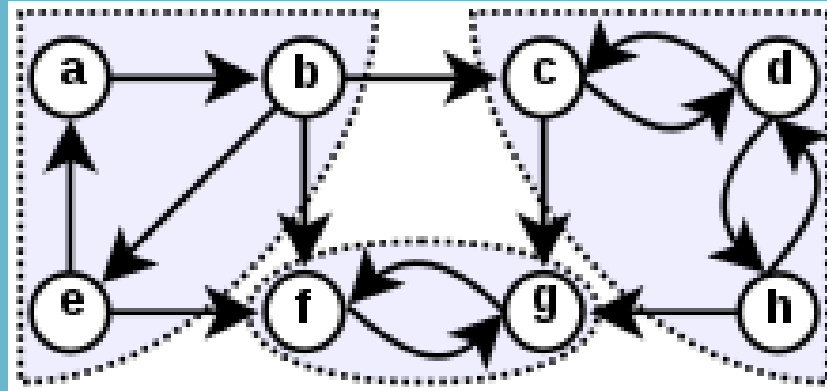
Example

How many connected components are in the graph?



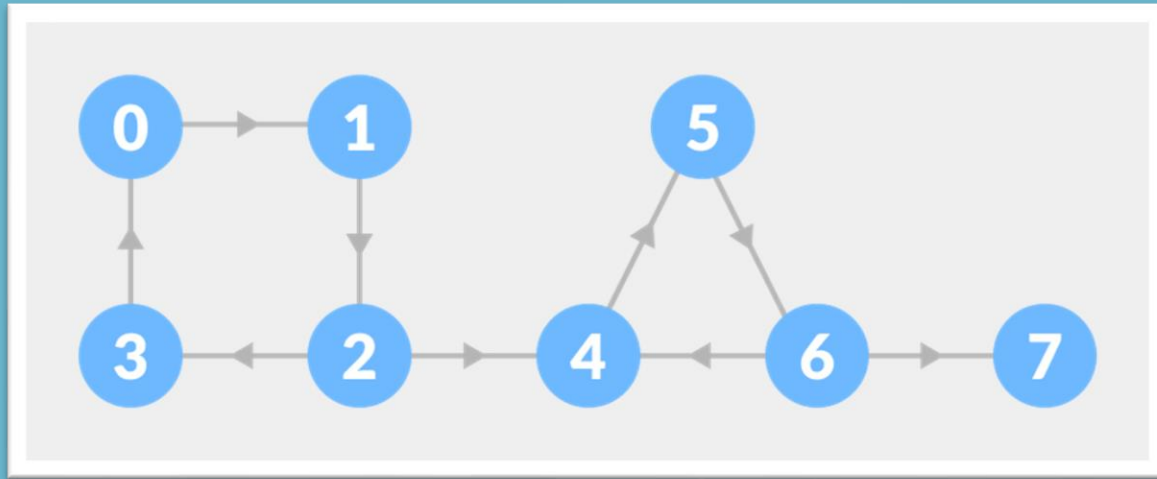
Components of Direct Graphs

- In a directed graph we can have connected components, but if we follow directions, then it may happen that we cannot reach all nodes (for example from f we can not reach all other nodes, but only g).
- A **strongly connected component** is a portion of a directed graph where there is a directed path between any two nodes (e.g. the nodes $\{a,b,c\}$ form a strongly connected component).
- The strongly connected components of a directed graph form a partition into subgraphs.



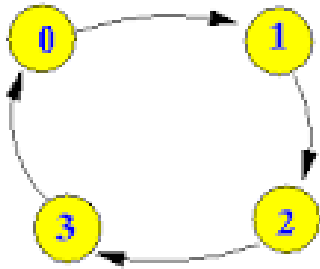
Example

How many strongly connected components are in the graph?

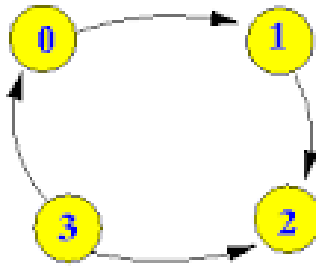


Strongly Connected Direct Graphs

A direct **graph** is said to be **strongly connected** if every vertex is reachable from every other vertex



Strongly connected



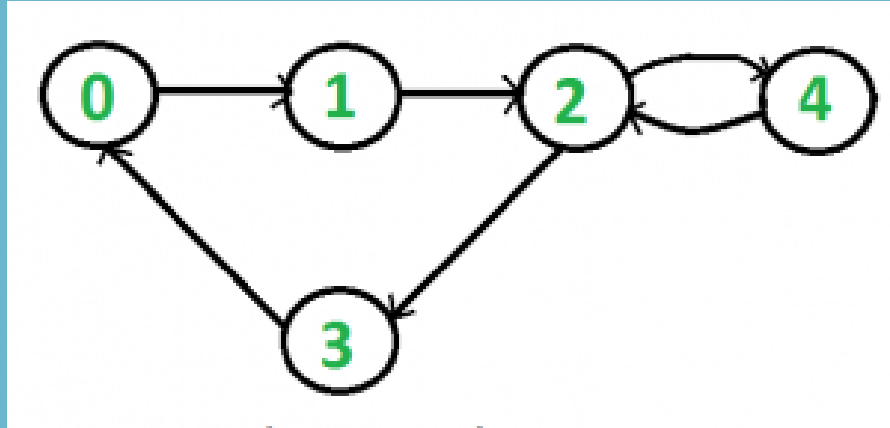
Not strongly connected

USE BFS :

- 1) Start from a node;
- 2) Obtain all reachable nodes and mark them;
- 3) Increment CC counter
- 4) Take next node not already marked, and start again

Example

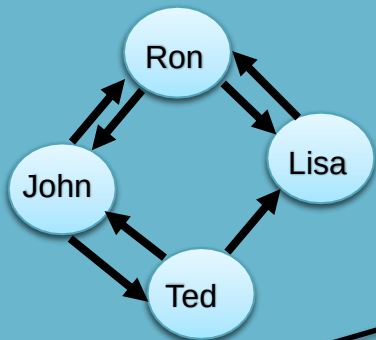
Is this graph strongly connected?



Conductance

Given a partition of a graph into two disjoint sets, $G1$ and $G2$, the conductance is the sum of the edges going from $G1$ to $G2$ divided by the minimum between the edges in $G1$ and in $G2$.

Attention: in an undirected graph we should consider both the directions for each edge

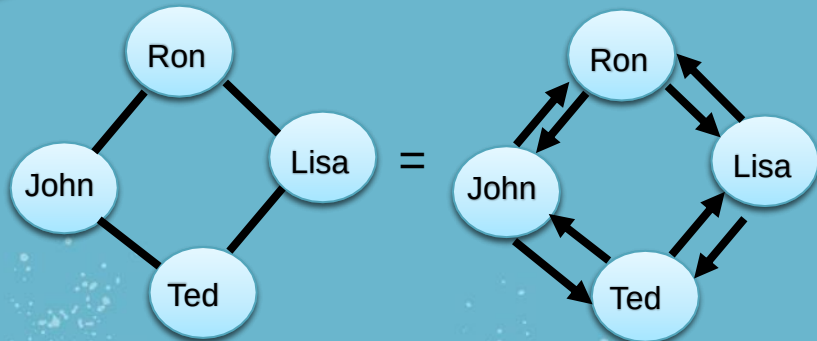


$G1=\{\text{John, Ted}\}; G2=\{\text{Ron, Lisa}\}$

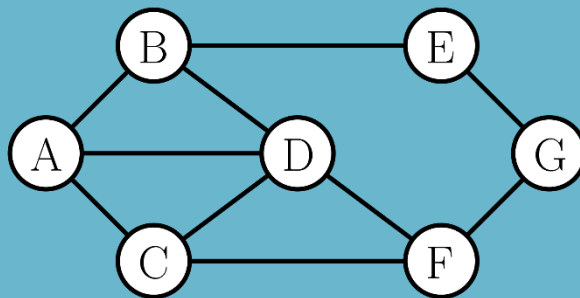
$$\text{Conductance} = \frac{2}{\min\{4,3\}} = \frac{2}{3} = 0.75$$

$G1=\{\text{John, Ted}\}; G2=\{\text{Ron, Lisa}\}$

$$\text{Conductance} = \frac{2}{\min\{4,4\}} = \frac{2}{4} = 0.5$$



Exercise

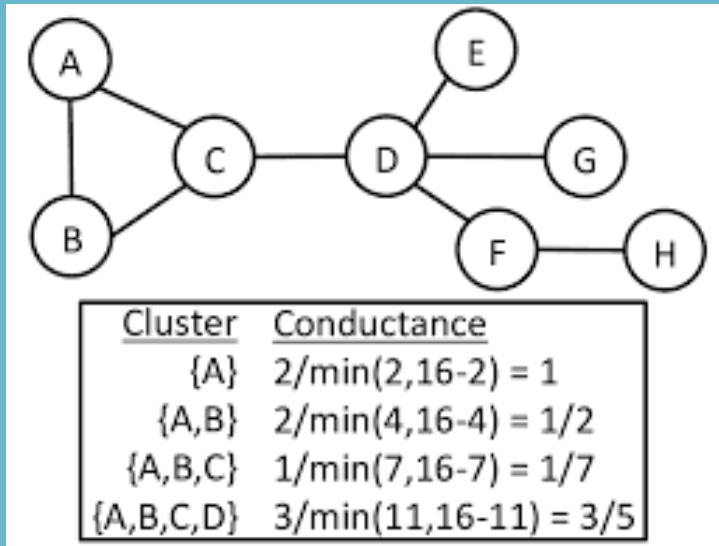


$$S_1 = \{A\} \Rightarrow \varphi(S_1) = \frac{3}{\min\{3,17\}} = 1$$

$$S_2 = \{A, B\} \Rightarrow \varphi(S_2) = \frac{4}{\min\{6,14\}} = \frac{2}{3}$$

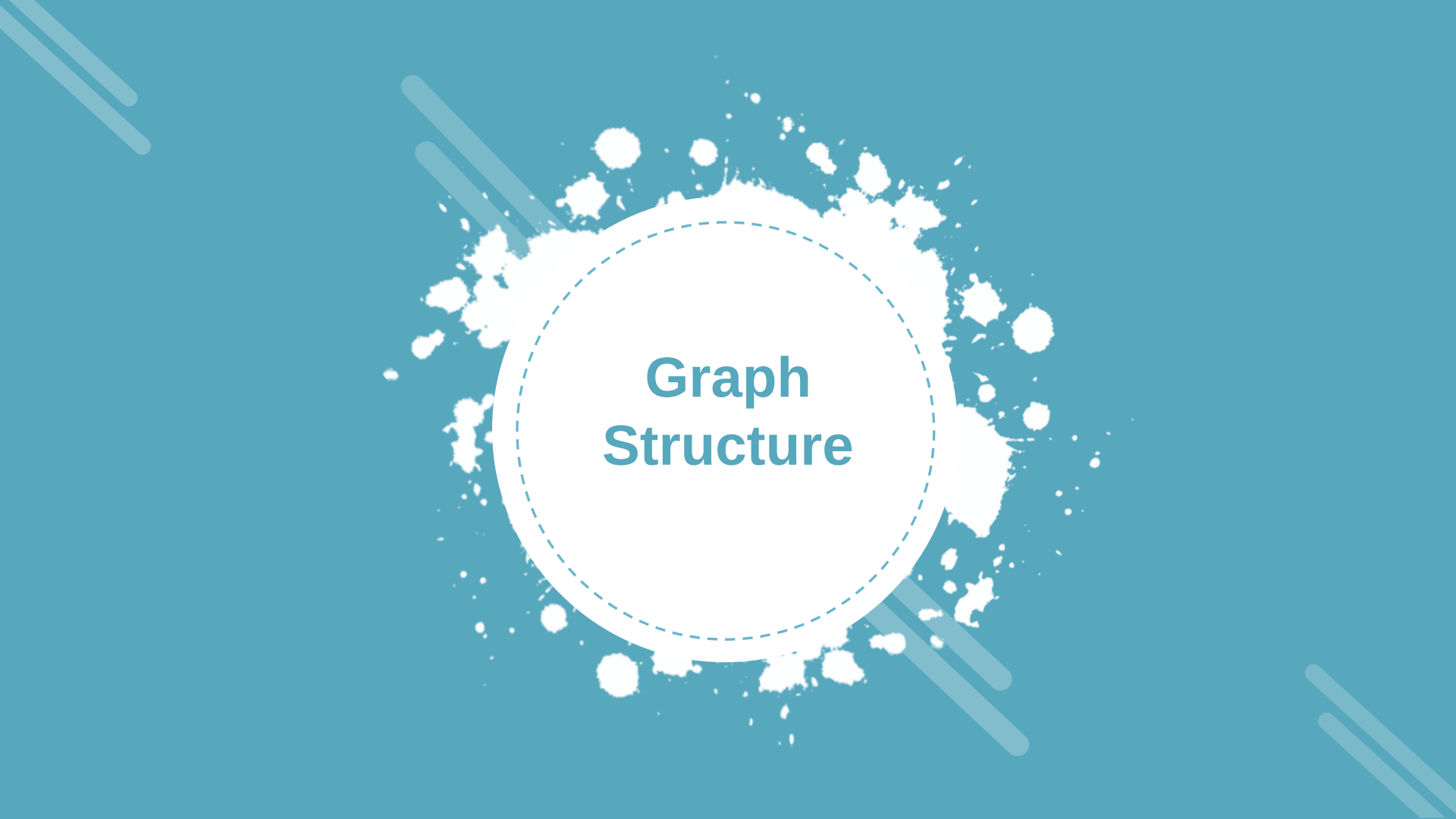
$$S_3 = \{A, B, C, D, E\} \Rightarrow \varphi(S_3) = \frac{3}{\min\{15,5\}} = \frac{3}{5}$$

Exercise





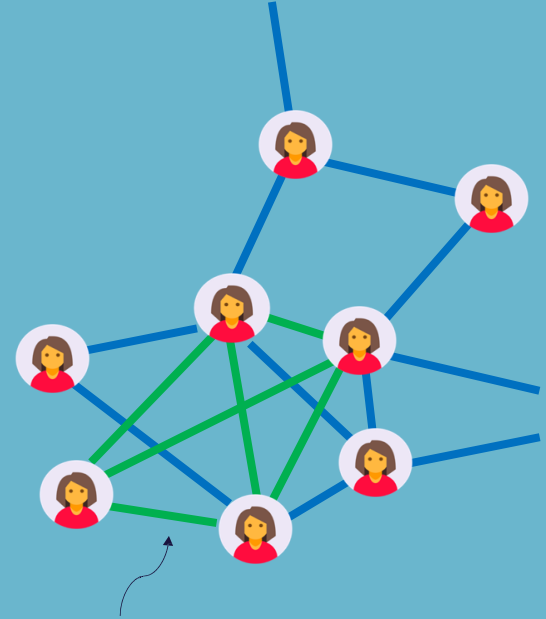
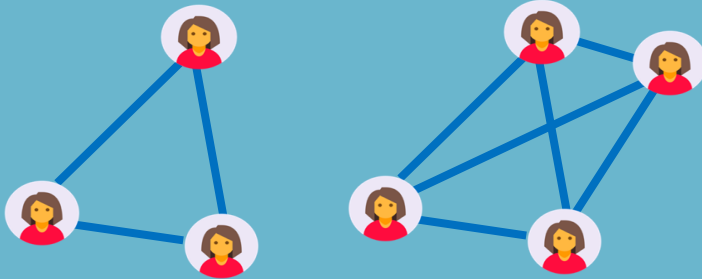
Section Break



Graph Structure

Cliques

Subset of vertices of an undirected graph such that every two distinct vertices in the clique are adjacent



Complete subgraphs:

A portion of the graph forming a clique, each node is connected to each other node

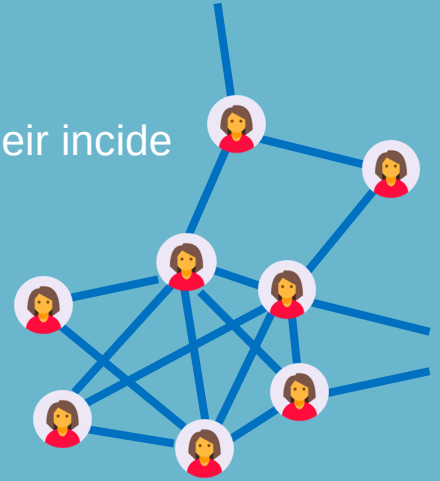
K-Core decomposition

K-Core decomposition is a graph algorithm that identifies the maximal subgraph of a graph where all nodes have at least degree k .

This maximal subgraph is called the k -core of the graph.

The algorithm iteratively removes nodes with degree less than k , and their incident edges, until no nodes with degree less than k remain.

```
Given  $K$ :  
Repeat:  
  for each node  $n$  :  
    if  $\text{degree}(n) < K$   
      : delete  $n$   
until all nodes have degree  $\geq K$ 
```



3-Core decomposition:

Every node has degree at least 3

Graph Structure Analysis

Characterize the structure of a graph:

- **Average Diameter L** : average length of the shortest paths connecting any two nodes
- **Clustering coefficient C** : the average local density

Small World Graphs have relatively small L & a relatively large C .

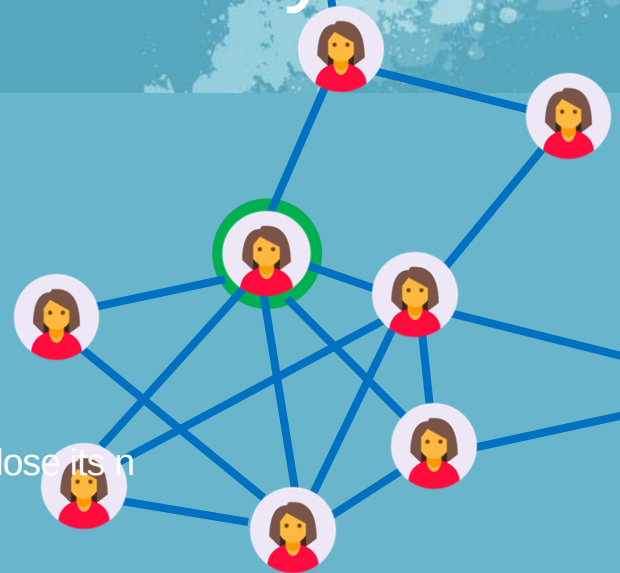
Clustering Coefficient: Local Density

How dense is the neighborhood of a node: The fraction pairs of neighbors of the node that are themselves connected

Density of a graph: fraction between number of edges and maximal number of edges

The **clustering coefficient** is Equivalent to the density of the subgraph when considering ONLY the neighbors of n (ignoring n)

The local clustering coefficient of a vertex (node) in a graph quantifies how close its neighbours are to being a clique (complete graph).



$$C_n = \frac{\text{\# edges between the neighbors of } n}{\text{degree}(n) * (\text{degree}(n) - 1)}$$

$$C_n = \frac{2 * (\text{\# edges between the neighbors of } n)}{\text{degree}(n) * (\text{degree}(n) - 1)}$$

Directed $D = \frac{|E|}{2 \binom{|V|}{2}} = \frac{|E|}{|V|(|V| - 1)}$

Undirected $D = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V| - 1)}$

Average clustering coefficient

How dense is the neighborhood of a node: The fraction pairs of neighbors of the node that are themselves connected

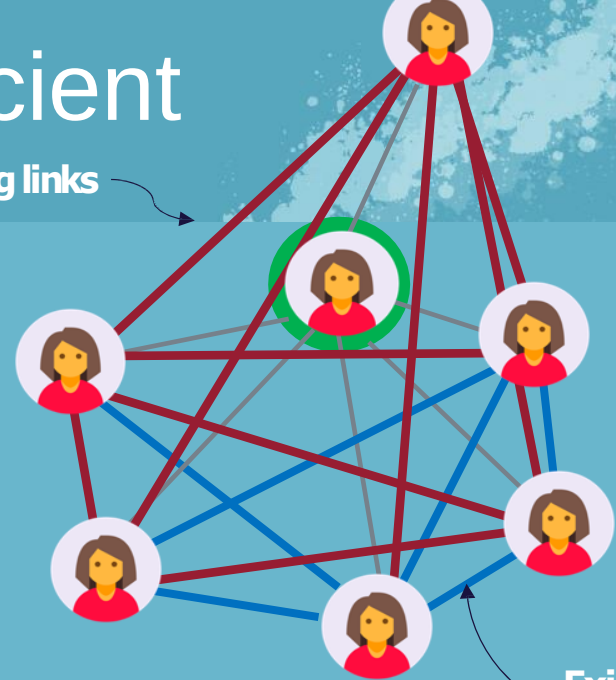
$$C_n = \frac{2 * \# \text{ edges between neighbors of } n}{\text{degree}(n) * (\text{degree}(n) - 1)}$$

Undirected

Clustering Coefficient
of the entire graph

$$C = \frac{1}{|N|} \sum_{n \in N} C_n$$

Missing links



Existing links

$$\text{Degree}(n) = 6$$

$$\text{Max connections} = \text{Degree}(n) *$$

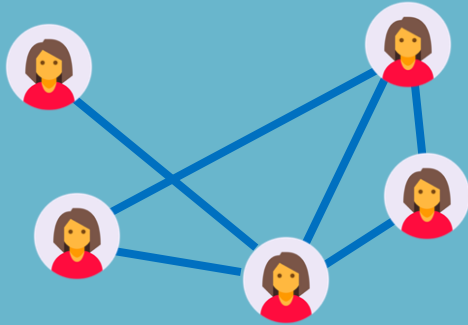
$$(\text{Degree}(n) - 1) = (6 * 5) = 30$$

$$\text{Existing links} = 6$$

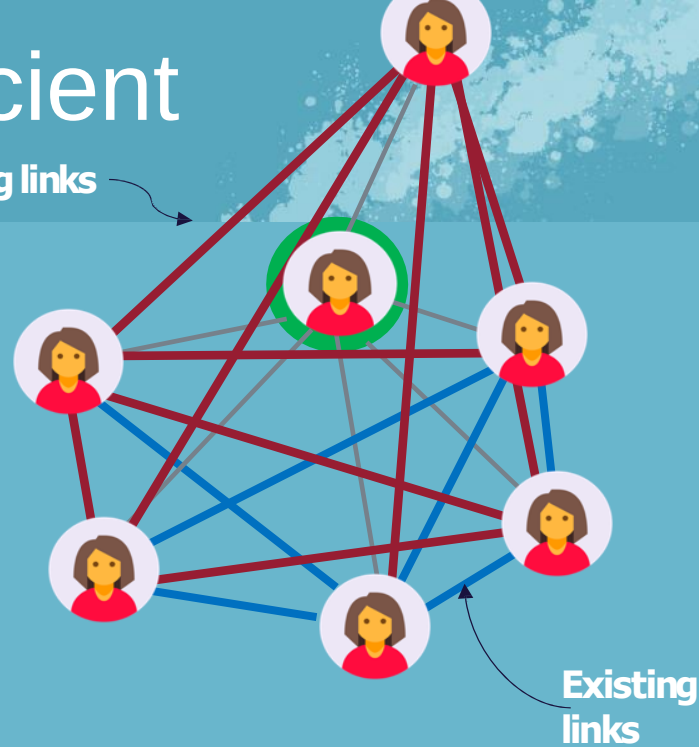
$$C_n = 2 * 6 / 30 = 0.4$$

Average clustering coefficient

Subgraph considering only the neighbors of n



Missing links



Wiener Index: Closeness of a graph

How tightly connected is a graph?

The Wiener index of a graph is a measure of its topological complexity, defined as the sum of all pairwise shortest path distances between nodes in the graph.

$$\sum_{(u,v) \in G} d(u, v)$$

$d(u, v)$ is the shortest-path distance





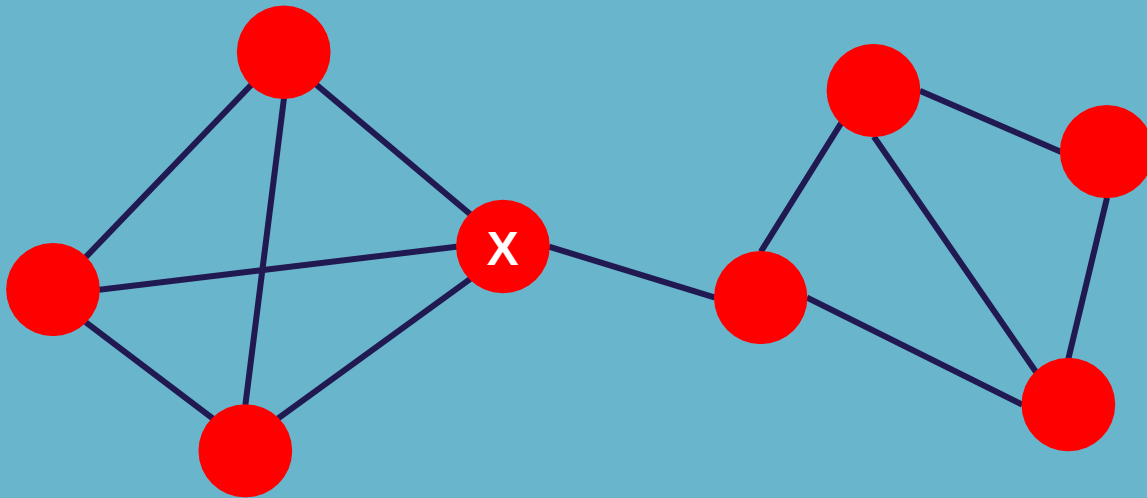
Centrality Measures

Importance of a Node

How important is a node in a graph?

Centrality intuition:

The importance of a node depends on its role in “keeping the graph connected”



Basic Centrality measures

Degree centrality: number of neighbors of node v

Closeness centrality: reciprocal of the total distance from a node v to all the other nodes in a network

$$C_c(v) = \frac{1}{\sum_{u \in V} \delta(u, v)}$$

$\delta(u, v)$ is the distance between node u and v .

Betweenness centrality [0-1]: ratio of the number of shortest paths passing through a node v out of all shortest paths between all node pairs in a network

$$C_B(v) = \sum_{s \neq t \neq v \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

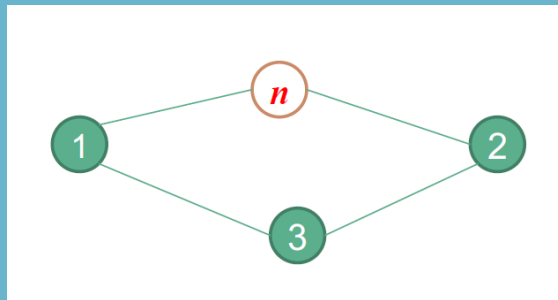
σ_{st} : number of shortest paths between node s and t

$\sigma_{st}(v)$: number of shortest paths passing on a node v out σ_{st}

These measure are meaningful only when referring to a connected graphs

Example

- For an undirected network with 4 node
- $\text{Degree}(n) = 2$ — $\text{AVG Degree} = 2$
- Closeness $C_C(n) = 1/(1+2+1) = 1/4$
- Shortest paths between all pairs:
 - $P(1, n) = [1, n];$
 - $P(1, 2) = [1, n, 2]; [1, 3, 2];$
 - $P(1, 3) = [1, 3];$
 - $P(2, n) = [2, n];$
 - $P(2, 3) = [2, 3];$
 - $P(3, n) = [3, 1, n]; [3, 2, n];$



- Shortest paths passing through n :
 - $[1, n];$
 - $[1, n, 2];$
 - $[2, n];$
 - $[3, 1, n]; [3, 2, n];$
- Betweenness $C_B(n) = 5/8$

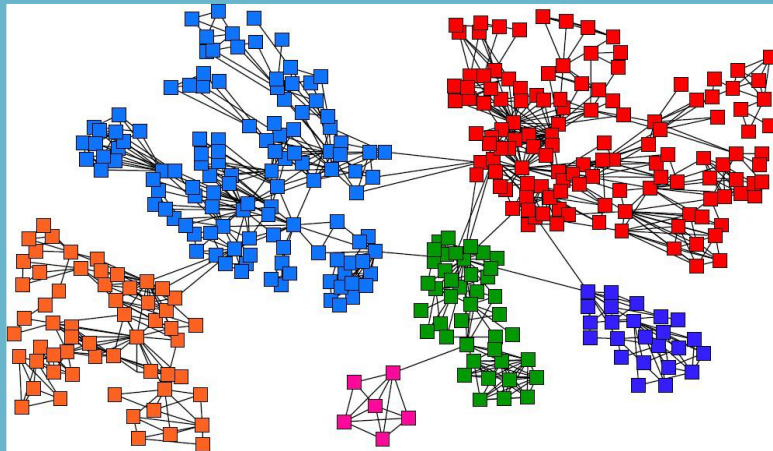
Community Detection

Community: portion of a graph with high internal connectivity (also called modules or clusters)

Divisive hierarchical clustering based on the notion of edge betweenness (Girvan-Newman Algorithm):

1. Calculate edge betweenness: find edges that are more “central” in the graph
 2. Delete high-betweenness edges: delete the edges with highest betweenness
 3. Connected components are communities
- Repeat until stopping condition

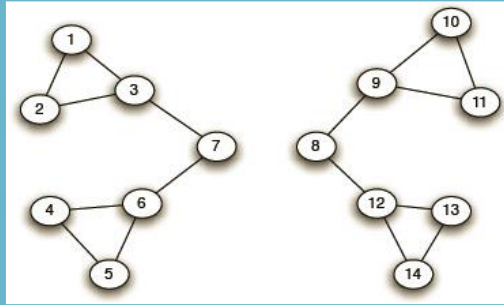
Quality of communities
Use the Conductance



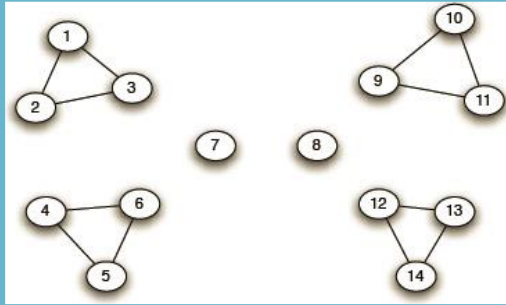
Community Detection

Divisive hierarchical clustering based on the notion of edge betweenness (Girvan-Newman Algorithm):

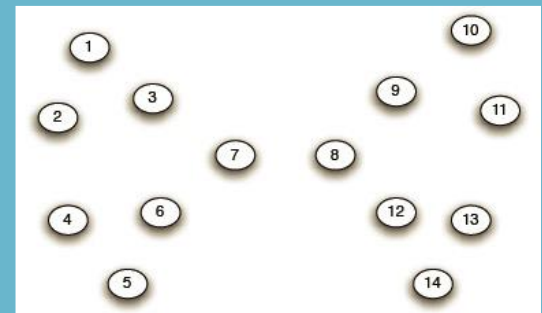
STEP 1



STEP 2

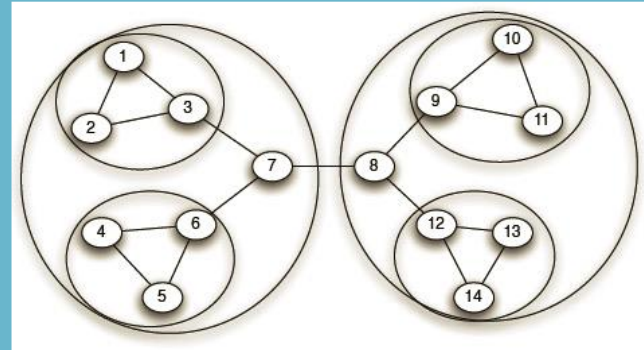


STEP3



Hierarchical network decomposition:

Not very effective for overlapping communities!



Community Detection - Girvan-Newman Algorithm

The Girvan-Newman algorithm for the detection and analysis of community structure relies on the iterative elimination of edges that have **the highest number of shortest paths** between nodes passing through them.

By removing edges from the graph one-by-one, the network breaks down into smaller pieces, so-called **communities**. The algorithm was introduced by Michelle Girvan and Mark Newman.

```
REPEAT
  LET n BE number of edges in the graph
  FOR i=0 to n-1
    LET B[i] BE betweenness centrality of edge i
    IF B[i] > max_B THEN
      max_B = B[i]
      max_B_edge = i
    ENDIF
  ENDFOR
  remove edge i from graph
UNTIL number of edges in graph is 0
```

<https://networkx.guide/algorithms/community-detection/girvan-newman/>

Graph Partitioning

Graph Cut: The number of edges that go from G_1 to G_2

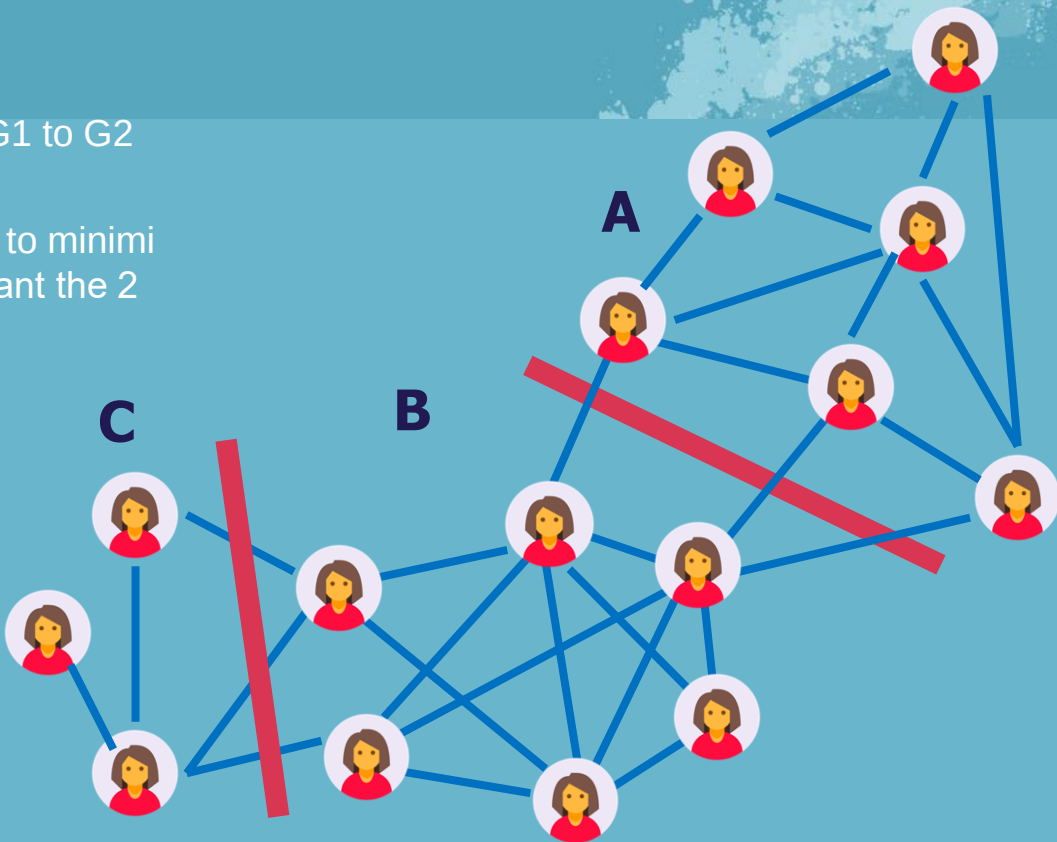
$$\mathbb{C} = |\text{Cut}(G_1, G_2)|$$

If we need to split the graph in 2 parts, we want to minimize the number of edges “break” and we also want the 2 parts to be “densely connected”

Conductance: given two portions of a graph G_1 and G_2 , the conductance is the sum of the edges going from G_1 to G_2 divided by the minimum between the edges in G_1 and in G_2

We want to minimize the following:

$$C(G_1, G_2) = \frac{|\mathbb{C}|}{\min\{(\mathbb{C} + |G_1|), (\mathbb{C} + |G_2|)\}}$$



Example

$$\mathbb{C} = |\text{Cut}(G_1, G_2)|$$

We want to minimize the following

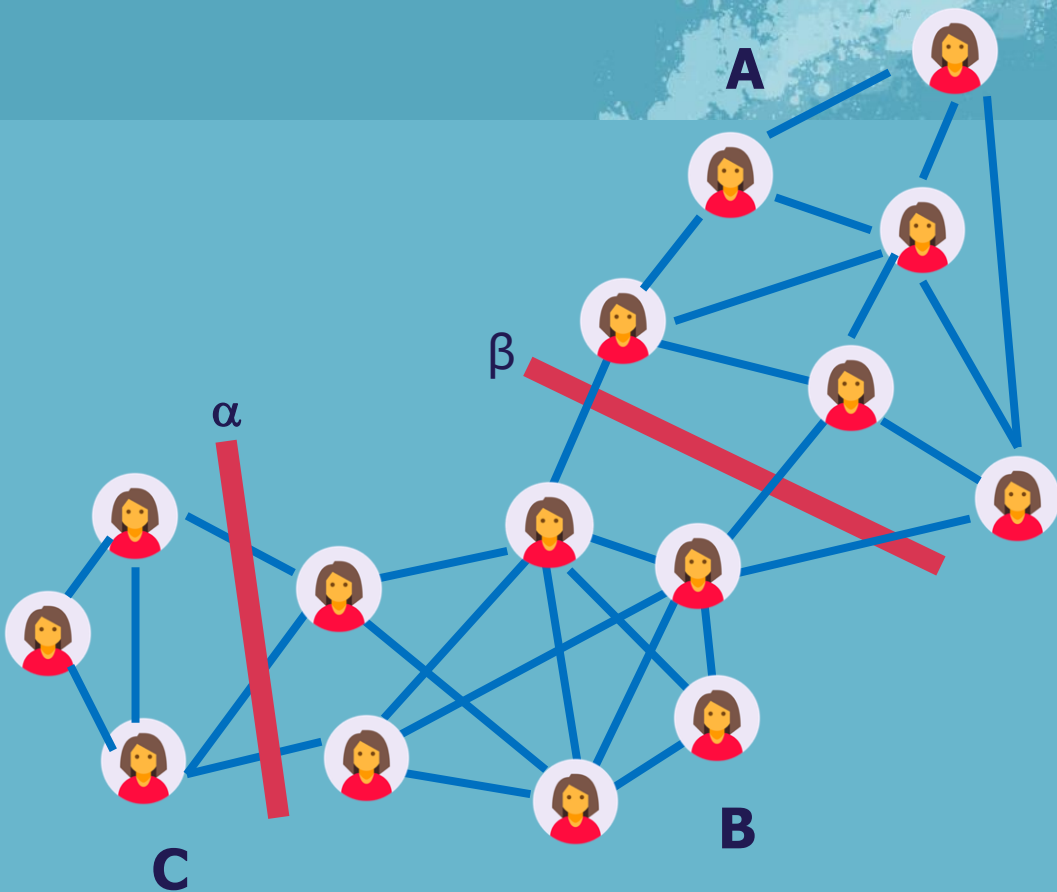
$$C(G_1, G_2) = \frac{|\mathbb{C}|}{\min\{(\mathbb{C} + |G_1|), (\mathbb{C} + |G_2|)\}}$$

$$|A| = 10 \quad |B| = 10 \quad |C| = 3$$

$$|\alpha| = 3 \quad |\beta| = 3$$

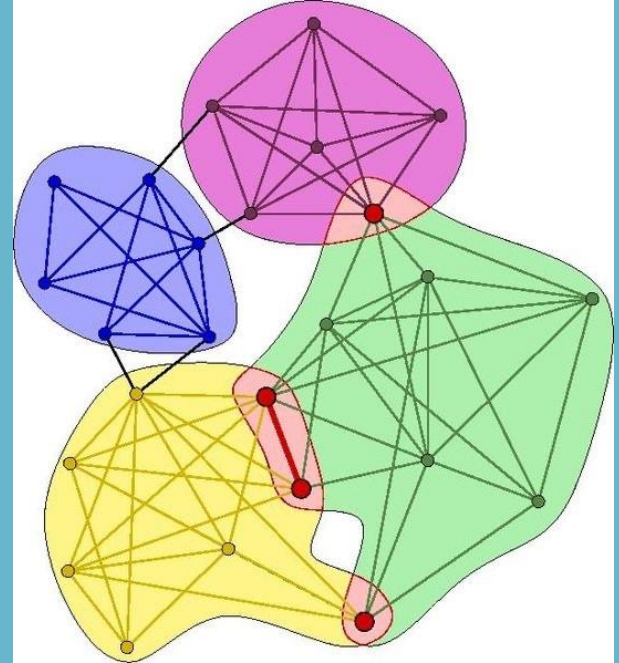
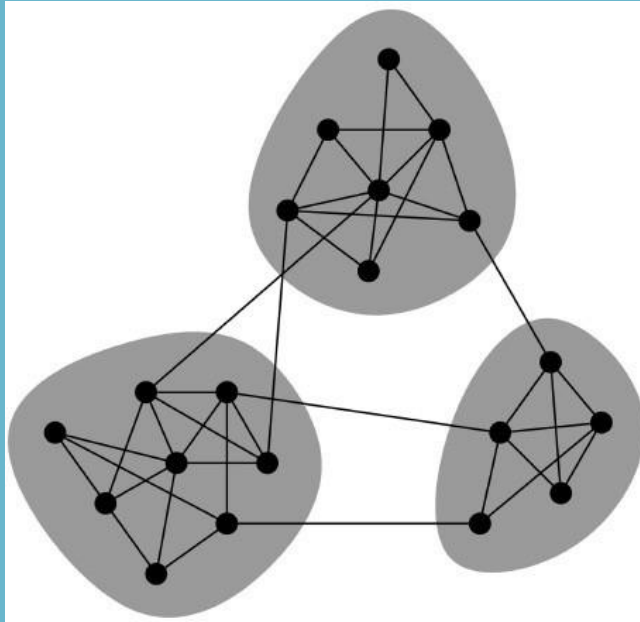
$$C(A, B + C) = \frac{3}{\min\{(3 + 10), (3 + 16)\}} \approx 0.231$$

$$C(A + B, C) = \frac{3}{\min\{(3 + 23), (3 + 3)\}} \approx 0.5$$



Non-overlapping vs. overlapping communities

Which one is Facebook?

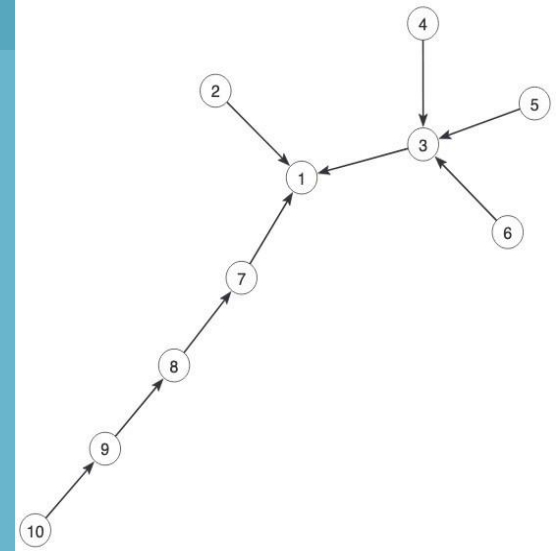


Importance in Directed Graphs: Proximity Prestige

Consider the importance of the node base on the “in-links”

Measures of prestige are usually computed for directed networks only, since for this measures the direction is important property of the relation.

Example: Persons, who are chosen as friends by many others have a special position – prestige in the group



Importance in Directed Graphs: Proximity Prestige

Consider the importance of the node base on the “in-links”

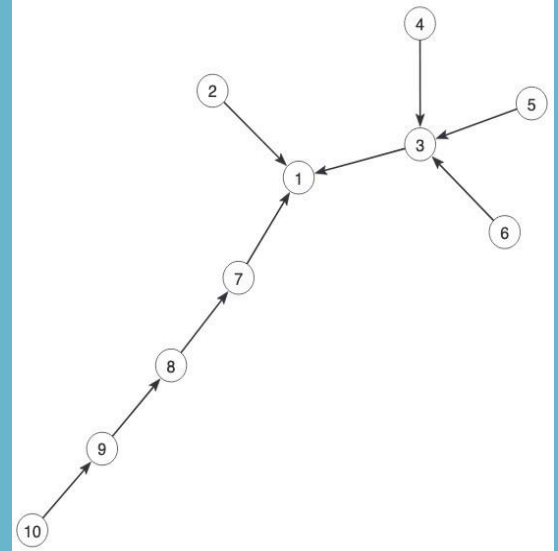
1. **In-Degree prestige:** number of incoming links of the node i : $\text{in-degree}(i)$
2. **Influence Domain:** number of nodes that can reach node i : $|I_i|$
3. **Proximity prestige:** proportional to the count of other nodes that can reach the node with a directed path and the distance of those paths

$$PP(i) = \frac{|I_i| / (n - 1)}{\sum_{j \in I_i} d(j, i) / |I_i|}$$

$|I_i|$ is the number of nodes that can reach i with a directed path

$(n-1)$ number of other nodes in the graph

$d(j, i)$ is the length of the directed shortest path



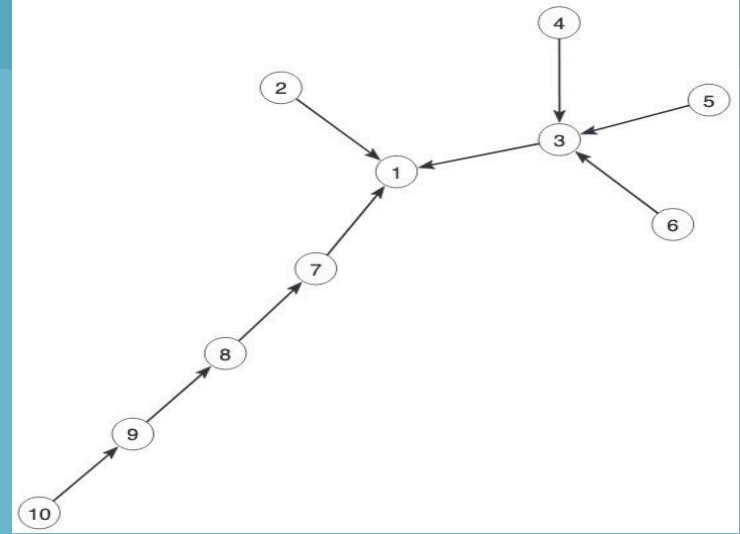
Importance in Directed Graphs: Proximity Prestige

$$PP(i) = \frac{|I_i| / (n - 1)}{\sum_{j \in I_i} d(j, i) / |I_i|}$$

$|I_i|$ is the number of nodes that can reach i with a directed path

$(n-1)$ number of other nodes in the graph

$d(j, i)$ is the length of the directed shortest path



$$\text{In-degree}(1) = 3$$

$$|I_1| = 9$$

$$PP(1) = \frac{9/9}{(1+1+2+2+2+1+2+3+4)/9} = \frac{1}{18/9} = \frac{1}{2} = 0.5$$

$$\text{In-degree}(8) = 1$$

$$|I_8| = 2$$

$$PP(8) = \frac{2/9}{(1+2)/2} = \frac{2/9}{3/2} = \frac{0.222}{1.5} = 0.148$$

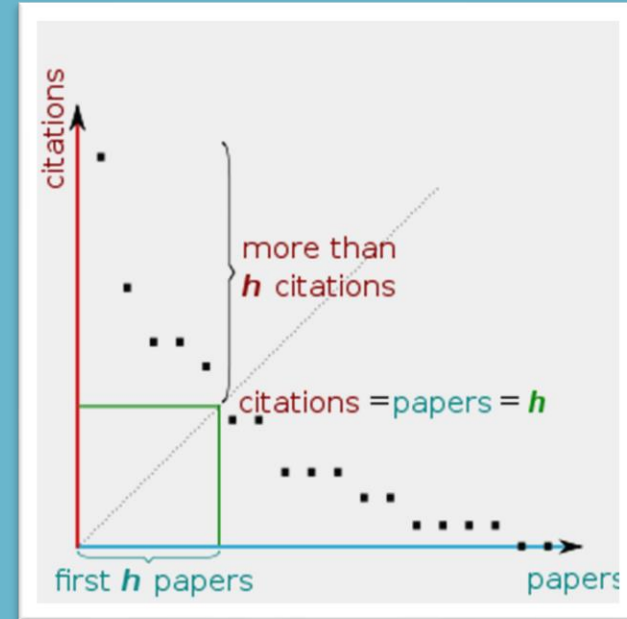
Measure Impact: H-Index

Consider “importance” of an author as balance between the “productivity” and the “impact”

1. **Productivity**: number of publications of an author
2. **Impact**: number of citations of an author's work
3. **H-Index**: the maximum value of h such that the given author/journal has published h papers that have each been cited at least h times

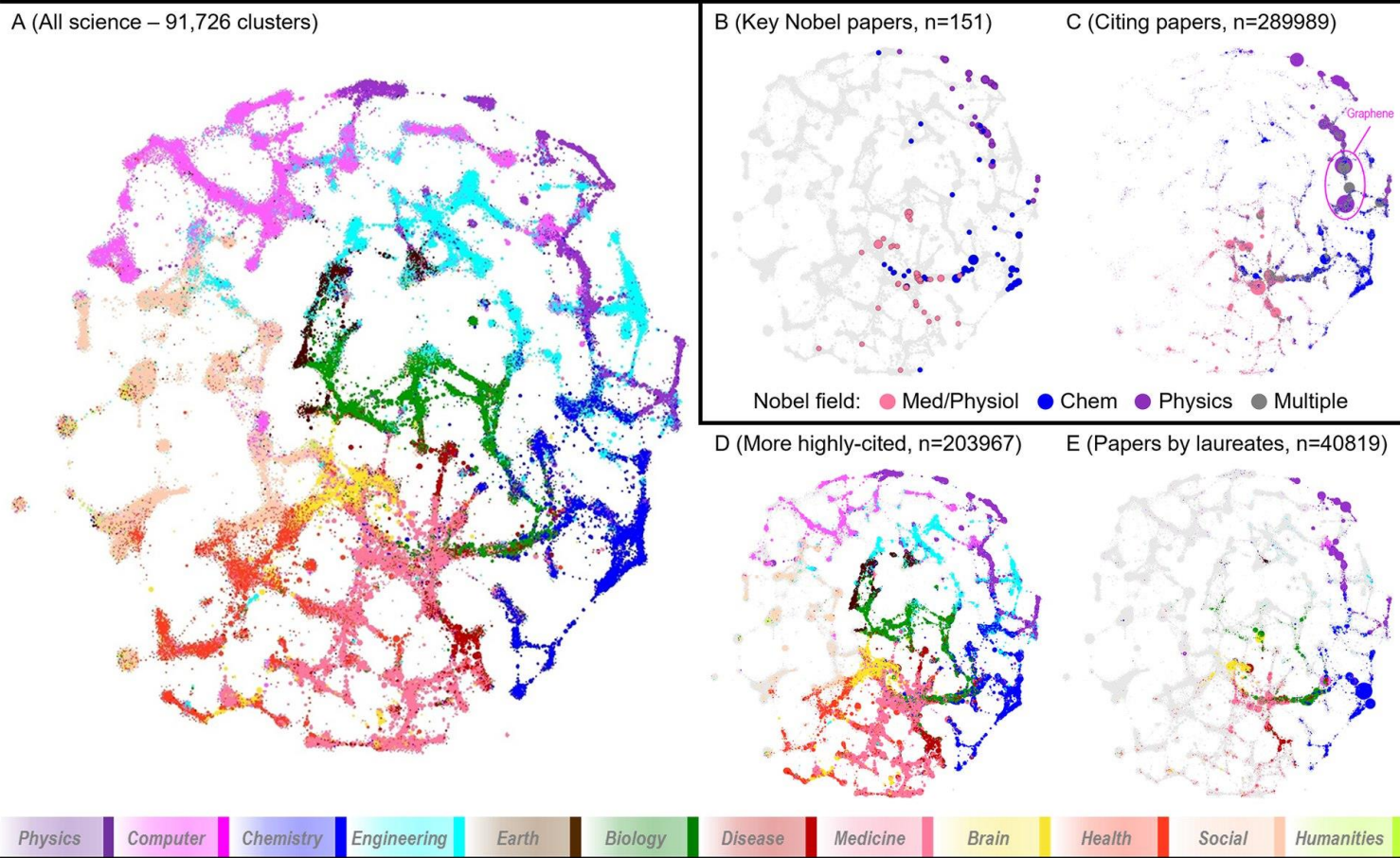
$Cit(A)=10, Cit(B)=8, Cit(C)=5, Cit(D)=4, Cit(E)=3 \rightarrow h\text{-index}=4$

$Cit(A)=25, Cit(B)=8, Cit(C)=5, Cit(D)=3, Cit(E)=3 \rightarrow h\text{-index}=3$



How do we translate it on a graph?

<https://en.wikipedia.org/wiki/H-index>



"**(A)** Map of science 91,726 clusters and associated large disciplines. The color legend is derived from the original construction of that map of science [15]). **(B)** The location of the key papers for Nobel prizes in Physics (purple), Chemistry (blue), and Physiology/Medicine (red) in 1995–2017. **(C)** The location of papers citing the key papers for Nobel prize s. Color legend as in panel B; grey color is used for areas that cite papers from two or more of three Nobel types. **(D)** The location of papers that have been more-cited than the r espective Nobel key paper even though they were published within one calendar year of the Nobel key paper. Color legend as in panel A. **(E)** the location of the entire career-long publication record (40,819 papers) published by scientists awarded Nobel prizes in Physics, Chemistry, and Physiology/Medicine 1995–2017. Color legend as in panel A."

